

GELİŞMİŞ RAG SİSTEMLERİ: KOD REHBERİ

LlamaIndex ve TruLens ile Adım Adım Kod Blokları ve Teknik Analiz

Bu kılavuz, DeepLearning.AI bünyesinde sunulan **Building and Evaluating Advanced RAG Systems** dersindeki uygulamalı kod örneklerini detaylı bir şekilde açıklamak amacıyla hazırlanmıştır. Çalışmada, geleneksel (baseline) RAG yapısının sınırlamaları ele alınarak, bu sınırlamaları aşmak üzere geliştirilen **Sentence-Window Retrieval** ve **Auto-Merging Retrieval** gelişmiş arama yöntemlerinin kod mimarisi ve mantığı satır satır incelenmiştir.

1. Temel (Baseline) RAG Kod Yapısı

Temel RAG sistemi dökümanları yükler, sabit boyutlu parçalara böler (chunking), bunları vektörleştirip indekse kaydeder ve doğrudan sorguya açar. Kod yapısı şu şekildedir:

```
# 1. Gerekli kütüphanelerin ve sınıfların içe aktarılması
from llama_index import SimpleDirectoryReader, VectorStoreIndex, ServiceContext
from llama_index.llms import OpenAI

# 2. Dokümanların yüklenmesi ve tek bir metin dökümanı olarak birleştirilmesi
documents = SimpleDirectoryReader(input_files=["./how_to_build_a_career_in_ai.pdf"]).load_data()
# Daha iyi metin bütünlüğü ve geçişleri için sayfalar birleştirilir
from llama_index.schema import Document
document_text = "\n\n".join([doc.text for doc in documents])
combined_document = Document(text=document_text)

# 3. Dil Modeli (LLM) ve Gömme (Embedding) Modeli Tanımlaması (ServiceContext)
llm = OpenAI(model="gpt-3.5-turbo", temperature=0.1)
service_context = ServiceContext.from_defaults(
    llm=llm,
    embed_model="local:BAAI/bge-small-en-v1.5"
)

# 4. Verilerin İndekslenmesi ve Sorgu Motorunun Oluşturulması
index = VectorStoreIndex.from_documents([combined_document], service_context=service_context)
query_engine = index.as_query_engine()

# 5. Sorgu Gönderme
response = query_engine.query("What are steps to take when finding projects?")
print(str(response))
```

Kod Bloklarının Detaylı Analizi:

- SimpleDirectoryReader ve Tek Doküman Birleştirme:** Kodda sayfalar halinde okunan metinler tek bir büyük Document nesnesinde birleştirilir. Bunun ana nedeni, gelişmiş arama yöntemlerinin (örneğin Sentence-Window) sayfa sınırlarında metin bütünlüğünü kaybetmesini engellemektir.
- ServiceContext:** LlamaIndex uygulamasının global yapılandırma merkezidir. Burada LLM olarak gpt-3.5-turbo ve embedding aracı olarak performansı yüksek lokal bir model olan BGESmall tanımlanmıştır.

- **VectorStoreIndex.from_documents:** Bu tek satırlık çağrı arka planda metni varsayılan boyutlarda (genellikle 1024 token) parçalara ayırır, vektörleştirir ve bellekte sorgulanabilir bir vektör uzayı oluşturur.

2. TruLens ile RAG Üçlüsü (Triad) Değerlendirmesi

Temel sistemin performansını ölçmek için TruLens kütüphanesi kullanılarak **RAG Triad** (Bağlam Uygunluğu, Groundedness ve Yanıt Uygunluğu) metrikleri kurulur. Değerlendirmeyi çalıştıran kod mimarisi şudur:

```
from trulens_eval import Tru, Feedback, TruLlama
from trulens_eval.feedback.provider.openai import OpenAI as TruOpenAI

tru = Tru()
tru.reset_database() # Geçmiş değerlendirme kayıtları temizlenir

# 1. Değerlendirme Sorularının Tanımlanması
eval_questions = [
    "What are the keys to building a career in AI?",
    "How can teamwork contribute to success in AI?",
    "What is the right AI job for me?" # Manuel eklenen soru
]

# 2. TruLens Recorder (Kaydedici) Kurulumu ve Metriklerin Bağlanması
# get_feedbacks helper işlevi arka planda Groundedness, Context & Answer Relevance'ı tanımlar
from utils import get_feedbacks
feedbacks = get_feedbacks()

tru_recorder = TruLlama(
    query_engine,
    app_id="Baseline RAG - V1",
    feedbacks=feedbacks
)

# 3. Sorguların Recorder Bağlamında Çalıştırılması
for question in eval_questions:
    with tru_recorder as recording:
        response = query_engine.query(question)
```

Önemli Kavramlar ve Değerlendirme Akışı:

- **tru.reset_database():** Her test çalıştırmasından önce veritabanını sıfırlayarak temiz bir izleme ve karşılaştırma ortamı sağlar.
- **tru_recorder (TruLlama):** Sorgu motorunun üzerine geçirilen bir 'izleyici' (wrapper) katmandır. Her sorunun girdi-çıktısını yakalar, harcanan token maliyetini ve gecikme süresini (latency) kaydeder ve LLM tabanlı hakem modelleri (Feedback functions) tetikler.
- **RAG Triad:** Bağlam uygunluğu (Context Relevance), dökümandan çekilen bilgilerin doğruluğu (Groundedness) ve nihai cevabın soruya odaklanma derecesini (Answer Relevance) sayısal olarak (0-1 arası) raporlar.

DEĞERLENDİRME ANALİZİ

Temel RAG test edildiğinde Answer Relevance ve Groundedness yüksek çıksa da, gereksiz büyük metin parçalarının çekilmesinden dolayı Context Relevance (Bağlam Uygunluğu) skoru düşük gelmektedir. Bu durum bizi gelişmiş tekniklere yönlendirir.

3. Sentence-Window Retrieval Kod Yapısı

Bu yöntemde dökümanlar tek tek cümlelere bölünerek indekslenir. Arama anında sadece en alakalı cümle bulunur (granüler arama), fakat sentez aşamasında bu cümlenin etrafındaki komşu cümleler (örneğin önündeki ve arkasındaki 3 cümle) birleştirilerek LLM'e daha geniş bir bağlam (context window) sunulur.

```
from llama_index.node_parser import SentenceWindowNodeParser
from llama_index.indices.postprocessor import MetadataReplacementPostprocessor
from llama_index.indices.postprocessor import SentenceTransformerRerank

# 1. Cümle Bölümleyici Tanımlanması (Her cümleyi ayırır ve etrafını saklar)
node_parser = SentenceWindowNodeParser.from_defaults(
    window_size=3, # Hedef cümlenin sağından ve solundan 3'er cümle ekle
    window_metadata_key="window",
    original_text_metadata_key="original_text"
)

# 2. İndeks Oluşturma (Cümle bazlı parçalama şemasıyla)
sentence_context = ServiceContext.from_defaults(
    llm=llm,
    embed_model="local:BAAI/bge-small-en-v1.5",
    node_parser=node_parser # Bu parser dökümanı cümlelere ayırır
)
sentence_index = VectorStoreIndex.from_documents([combined_document], service_context=sentence_context)

# 3. Postprocessor (Metadata Değiştirici) Tanımlama ve Sorgu Motoru
# Arama yapıldıktan hemen sonra bulunan cümlenin yerine 'window' içeriğini (geniş bağlamı) yerleştirir
postprocessor = MetadataReplacementPostprocessor(target_metadata_key="window")

# İsteğe bağlı reranking (yeniden sıralama) adımı
rerank = SentenceTransformerRerank(top_n=2, model="BAAI/bge-reranker-large")

query_engine = sentence_index.as_query_engine(
    similarity_top_k=2,
    node_postprocessors=[postprocessor, rerank]
)
```

Kritik Bileşenlerin İşlevi:

- **SentenceWindowNodeParser:** Belgeyi cümle sınırlarından böler. Her bir cümlenin üst verisine (metadata), o cümlenin etrafındaki window_size kadar komşu cümleyi birleştirerek kaydeder.
- **MetadataReplacementPostprocessor:** RAG akışındaki en kritik adımdır. Vektör arama sadece en alakalı cümleyi (küçük, gürültüsüz veri) getirir. Ancak LLM'e göndermeden hemen önce bu postprocessor devreye girerek, o küçük cümlenin yerine saklanan geniş 'window' metnini koyar. Böylece LLM konsepti tam olarak kavrar.
- **SentenceTransformerRerank:** Arama sonucunda getirilen en iyi adayları, daha güçlü bir lokal model kullanarak en alakalı olandan başlayarak yeniden sıralar (Re-ranking). Bu da sentez kalitesini maksimize eder.

4. Auto-Merging Retrieval Kod Yapısı

Auto-Merging, dökümanı hiyerarşik bir ağaç yapısında böler. En üstte büyük ebeveyn parçaları (parent), onların altında ise küçük çocuk parçaları (child) bulunur. Vektör araması çocuk parçalarında yapılır. Eğer bir ebeveyne ait çocuk parçalarının çoğunluğu bulunursa, sistem bu parçaları otomatik olarak birleştirir ve LLM'e doğrudan büyük ebeveyn metnini gönderir.

```
from llama_index.node_parser import HierarchicalNodeParser, get_leaf_nodes
from llama_index.storage import StorageContext
from llama_index.retrievers import AutoMergingRetriever
from llama_index.query_engine import RetrieverQueryEngine

# 1. Hiyerarşik Bölümleyici Tanımlanması (Ebeveyn ve Çocuk ilişkisi)
node_parser = HierarchicalNodeParser.from_defaults(
    chunk_sizes=[2048, 512, 128] # 3 katmanlı hiyerarşik yapı
)
nodes = node_parser.get_nodes_from_documents([combined_document])

# En alt katmandaki yaprak (leaf/child) düğümler aranacak nesnelerdir
leaf_nodes = get_leaf_nodes(nodes)

# 2. StorageContext ve İndeksleme
storage_context = StorageContext.from_defaults()
storage_context.docstore.add_documents(nodes) # Tüm hiyerarşik ağaç saklanır

auto_merging_index = VectorStoreIndex(
    leaf_nodes, # Sadece en alt yaprak düğümler vektörleştirilir ve aranır
    storage_context=storage_context,
    service_context=service_context
)

# 3. Auto-Merging Retriever ve Sorgu Motoru Kurulumu
base_retriever = auto_merging_index.as_retriever(similarity_top_k=12)
retriever = AutoMergingRetriever(
    base_retriever,
    storage_context,
    simple_ratio=0.5 # Çocukların %50'si gelirse ebeveyn ile birleştir
)

query_engine = RetrieverQueryEngine.from_args(retriever, service_context=service_context)
```

Auto-Merging Akış Mantığı ve Log Analizi:

- **HierarchicalNodeParser:** Kodda görülen [2048, 512, 128] dizisi, metnin önce 2048 karakterlik devasa bloklara, ardından bunların 512 karakterlik ebeveynlere ve en son 128 karakterlik çocuk bloklara bölüneceğini ifade eder.
- **Arama ve Birleştirme Kararı:** Arama 128 karakterlik yaprak düğümlerde (leaf_nodes) yapılır. Eğer 512 karakterlik bir bloğun altındaki 4 çocuktan en az 2'si (simple_ratio=0.5 gereği) bulunursa, arama motoru bu çocukları siler ve yerine ebeveyni (512 tokenlık bloğu) koyar.
- **Konsol Logları (Log Output):** Kod çalışırken terminalde Merging 3 nodes into parent node. logunun görülmesi, çocuk düğümlerin başarıyla tespit edilip ebeveyn düğüm altında birleştirildiğini doğrular.

Tekniklerin Karşılaştırmalı Özeti

Metot / Teknik	Arama Katmanı (Retrieval)	Sentez Katmanı (LLM Context)	Temel Avantajı
Baseline RAG	Standart Chunk (1024 token)	Aynı Chunk (1024 token)	Hızlı ve kurulumu en basit yapı.
Sentence-Window	Tek Cümle (Granüler)	Cümle + Çevre Pencere	En yüksek Groundedness, gürültüsüz arama.
Auto-Merging	Küçük Çocuk Parçaları (128)	Büyük Ebeveyn Blokları (512)	Hiyerarşik bütünlük ve bağlam derinliği.